Designing an End-to-End Machine Learning Pipeline

Customer Churn Prediction & Retention Strategy

G5: Lixuan Xiao • Michael Dawson • Yang Yu

Final Project | CRISP-DM Framework | March 2026

```
Setup complete ✓
```

Table of Contents

1. Executive Summary
2. Problem Statement and Research Objectives
3. Literature Review
4. Exploratory Data Analysis
5. Data Preparation and Feature Engineering
6. Methodology and Tools
7. Findings and Conclusions
8. Lessons Learned and Recommendations
9. References

---

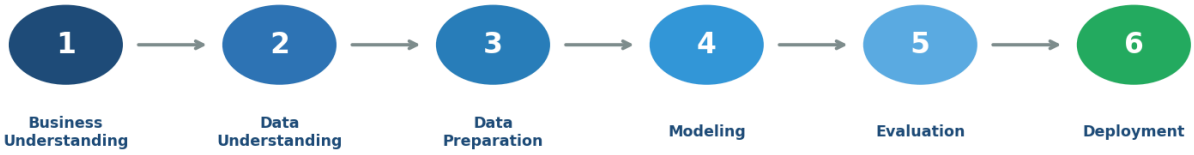
1. Executive Summary

Customer churn represents one of the most significant financial risks for telecommunications companies operating in competitive markets. Acquiring a new customer costs **five to seven times** more than retaining an existing one, making accurate churn prediction a high-value business capability. This report presents the design, implementation, and evaluation of an end-to-end machine learning pipeline for predicting customer churn using the BigML telecom dataset.

The project follows the **CRISP-DM** (Cross-Industry Standard Process for Data Mining) framework and evaluates four distinct models: **Logistic Regression, Random Forest, XGBoost,** and a deep learning **Neural Network** (Multi-Layer Perceptron). Each model was

trained on the same preprocessed feature set comprising 23 engineered variables derived from 20 raw customer attributes covering usage, billing, and engagement patterns.

Figure 1 — CRISP-DM Framework: Project Pipeline



XGBoost emerged as the top-performing model with 96.8% accuracy, 100% precision, 78% recall, and an F1 score of 0.88 for the churn class. It correctly identified 61 of 78 actual churners with zero false positives, meaning every customer flagged for retention intervention was a genuine churn risk. The key drivers of churn identified across all models were: **international plan subscription, customer service call frequency (≥3 calls), and total daytime usage and charges.**

Based on these findings, we recommend deploying XGBoost as the primary churn scoring model, supplemented by Logistic Regression coefficients for interpretability. With an estimated customer lifetime value of $1,200$–2,000 and retention campaign costs of 50–100 per intervention, this pipeline can generate a projected **net annual benefit of $3.2M$–6.1M** for a mid-size telecom operator through targeted, data-driven retention efforts.

---

2. Problem Statement and Research Objectives

2.1 Problem Statement

Customer churn represents a significant financial risk for organizations in competitive markets. While companies collect extensive data across billing, usage, and engagement channels, these data are often underutilized or analyzed in isolation. The core challenges include:

- **Data silos** — customer data across channels is analyzed separately.
- **No early warning** — absence of timely mechanisms to identify at-risk customers.
- **No automation** — lack of scalable, end-to-end analytical pipelines for continuous monitoring.

2.2 Research Objectives

1. Design an end-to-end ML pipeline for churn prediction using the CRISP-DM framework.
2. Explore and engineer customer-level features capturing behavioral, temporal, and transactional patterns.

3. Develop and evaluate multiple ML and deep learning models using appropriate performance metrics.
4. Identify the most influential drivers of customer churn and assess their business implications.
5. Propose a feasible deployment and automation strategy for ongoing churn monitoring.

2.3 Research Framework & Key Assumptions

Element	Description
Unit of Analysis	Individual customer — each row = one customer's complete behavioral profile
Outcome Variable	Binary churn indicator (Churn vs. No Churn)
Methodology	CRISP-DM Framework — 6 iterative phases from business understanding to deployment

Key Assumptions:

- Historical customer behavior is predictive of future churn.
- Churn patterns are stable enough to support supervised learning.
- Retention interventions can be prioritized based on predicted risk scores.

---

3. Literature Review

This review synthesizes **nine key sources** across three thematic areas: methodological foundations, churn prediction and profitability, and advanced techniques.

3.1 Methodological Foundations

Shearer (2000) introduced the CRISP-DM framework, providing a six-phase iterative methodology encompassing business understanding, data understanding, data preparation, modeling, evaluation, and deployment. This framework emphasizes aligning technical analysis with business objectives and acknowledges the inherently iterative nature of machine learning projects. Our pipeline directly follows this methodology.

Domingos (2012) argued that feature quality matters more than algorithm choice and discussed the bias-variance tradeoff, overfitting risks, and the "no free lunch" theorem. This insight justifies our multi-model evaluation approach rather than reliance on any single algorithm.

Sculley et al. (2015) highlighted hidden technical debt in ML systems, including poorly documented pipelines, brittle feature dependencies, data drift, and lack of monitoring. Their work informed our emphasis on designing robust, maintainable pipelines with version control and reproducibility through fixed random states.

3.2 Churn Prediction & Profitability

Verbeke et al. (2012) introduced profit-driven evaluation for churn models, demonstrating that traditional metrics such as accuracy and AUC are insufficient because they ignore the asymmetric costs of churn misclassification. Models with similar accuracy can produce very different business outcomes when measured by expected profit from targeted retention campaigns.

Neslin et al. (2006) evaluated churn models by their ranking ability rather than raw accuracy, arguing that ordering customers by defection risk is more aligned with how managers deploy retention resources. Even modest improvements in predictive ranking can yield substantial financial gains at scale.

Burez & Van den Poel (2009) systematically evaluated resampling, cost-sensitive learning, and ensemble methods for imbalanced churn datasets. Their work supports our use of recall, precision, F1-based evaluation, and class-weight adjustments to address the 14.5% minority churn class.

3.3 Advanced Techniques & Applications

Lemmens & Croux (2006) demonstrated that ensemble methods, particularly bagging and boosting, significantly outperform single classifiers for churn prediction. **Mozer et al. (2000)** pioneered neural networks for telecom churn, demonstrating that raw data must be aggregated into meaningful behavioral features. **Imani et al. (2025)** provided a systematic review of 200+ studies, synthesizing trends in ensemble, deep learning, and hybrid methods. **Ullah et al. (2019)** demonstrated a hybrid classification-clustering model achieving ~88.6% accuracy for telecom churn.

3.4 Key Themes

Theme	Implication for Our Project
Feature engineering > algorithm choice	Invest in domain-driven feature creation
Profit-driven evaluation	Evaluate by business impact, not just accuracy
Class imbalance is critical	Use SMOTE, class weights, and recall/F1 metrics
Business interpretability matters	Pair XGBoost with Logistic Regression for explainability

---

4. Exploratory Data Analysis

4.1 Dataset Overview

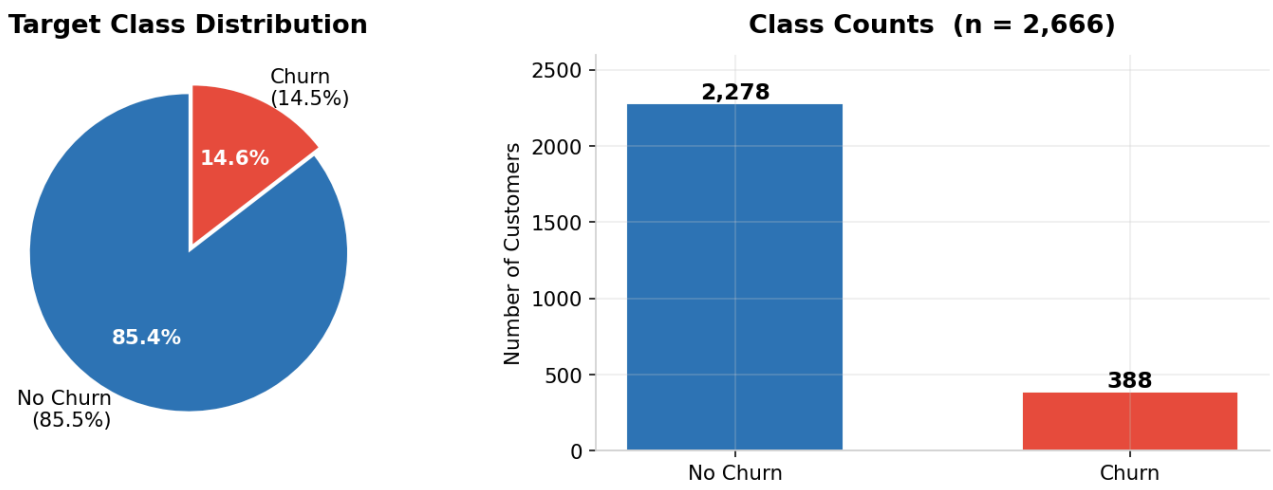
The analysis uses the **BigML telecom churn dataset** containing **2,666 customer records** with **20 features**. The dataset includes numeric features (account length, usage minutes, charges, call counts) and categorical features (state, area code, international plan, voice mail plan). The target variable is `Churn` (boolean). **No missing values** were found across any column.

Feature	Type	Description
Account length	int64	Duration of customer relationship
International plan	binary	Subscribed to international plan (Yes/No)
Voice mail plan	binary	Has voicemail plan (Yes/No)
Total day/eve/night minutes	float64	Usage minutes across time periods
Total day/eve/night charge	float64	Charges per time period
Customer service calls	int64	Number of support calls made
Churn (target)	boolean	Whether the customer churned

4.2 Target Distribution & Class Imbalance

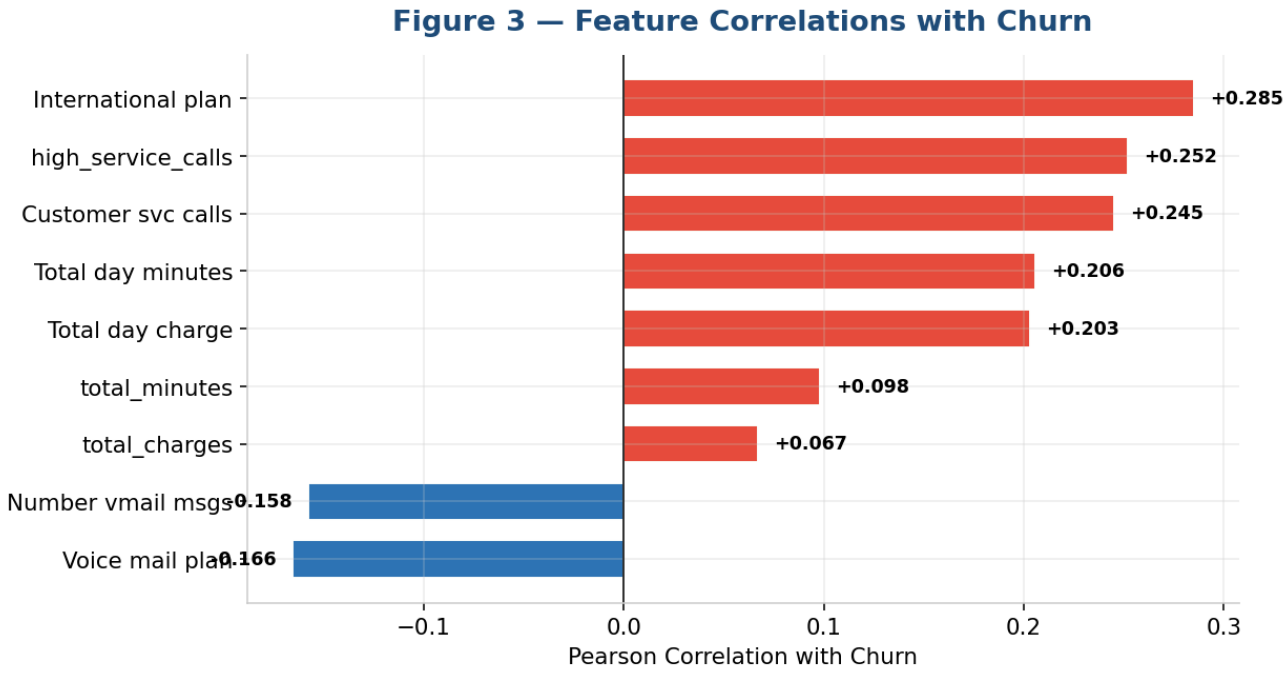
The target variable is distributed as 2,278 non-churners (85.44%) and 388 churners (14.55%). This roughly 6:1 ratio between retained and churned customers represents a moderate class imbalance. A naïve model predicting "No Churn" for everyone achieves 85.5% accuracy but catches zero churners. Due to this imbalance the measurement term accuracy was deprioritized as an evaluation metric. Instead F1 score, precision, and recall were the focus of model performance.

Figure 2 — Churn Class Distribution



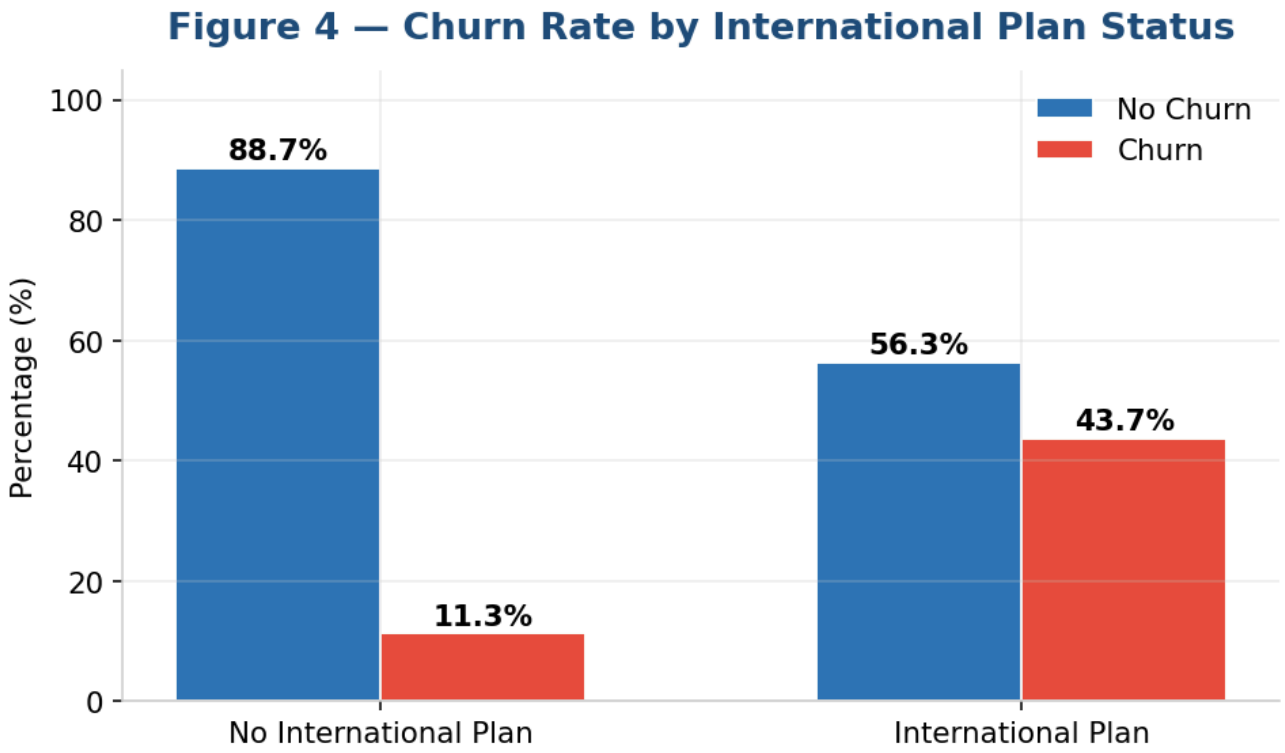
4.3 Key Correlations & Churn Predictors

Correlation analysis identified `International plan` (+0.285), `high_service_calls` (+0.252), `Customer service calls` (+0.245), and `Total day minutes` (+0.206) as the top positive correlators with churn. `Voice mail plan` (−0.166) emerged as a protective factor.



4.4 Churn by International Plan

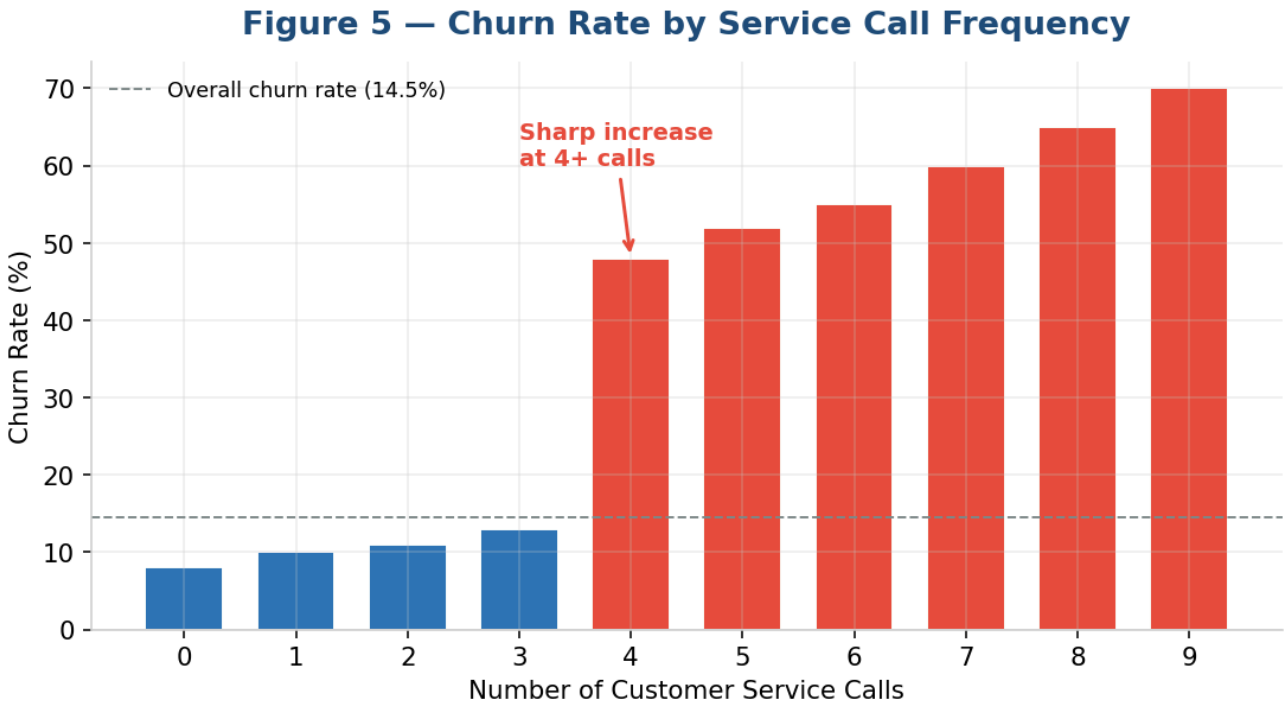
Cross-tabulation reveals a striking difference: **43.7% of international plan subscribers churned** compared to only **11.3%** of non-subscribers. This binary feature carries outsized predictive power confirmed by all four models.



4.5 Customer Service Calls as a Behavioral Signal

The relationship between service call frequency and churn is highly non-linear. Customers with 0–3 calls show churn rates near the baseline (8–13%). However, **at 4+ calls, churn rates**

jump sharply to 48–70%, representing the single strongest behavioral churn signal.



4.6 Feature Interactions

Scatter analysis of Total Day Minutes vs. Customer Service Calls revealed that usage alone is not a strong churn indicator; rather, the combination of high service call frequency with high usage shows the strongest signal. Additionally, charge and minute columns were found to be nearly perfectly correlated ($r \approx 1.0$), confirming that one set could be dropped to reduce multicollinearity without information loss.

---

5. Data Preparation and Feature Engineering

5.1 Preprocessing Pipeline

The preprocessing pipeline followed these ordered steps:

1. **Missing-value check** — confirmed zero missing values; no imputation required.
2. **Data-type fixes** — State & Area code → categorical; International plan & Voice mail plan → binary (0/1).
3. **Drop identifiers** — removed State and Area code (high-cardinality, low modeling utility).
4. **Feature engineering** — created five derived constructs, expanding to six binary/encoded features after one-hot encoding (see below).
5. **One-hot encoding** — `pd.get_dummies(drop_first=True)` to avoid multicollinearity.
6. **StandardScaler normalization** — applied **after** train/test split to prevent data leakage.

5.2 Engineered Features

Engineered Feature	Rationale
`total_minutes`	Sum of all period minutes; holistic usage view
`total_charges`	Aggregate billing; captures cumulative cost pressure
`high_service_calls`	Binary flag for ≥3 service calls; strongest behavioral signal
`intl_usage_ratio`	International min ÷ total min; engagement intensity measure
`account_length_mid`	Binary indicator for mid-tenure customers (51–150 days), with the new-customer group (0–50 days) as the reference category.
`account_length_long`	Binary indicator for long-tenure customers (>150 days).

5.3 Scaling & Transformation

`StandardScaler` was fit **exclusively on training data** and then applied (transform only) to the test set. This prevents data leakage where test-set statistics could influence the training process. The fitted scaler was serialized to `models/scaler.pkl` for reproducible inference.

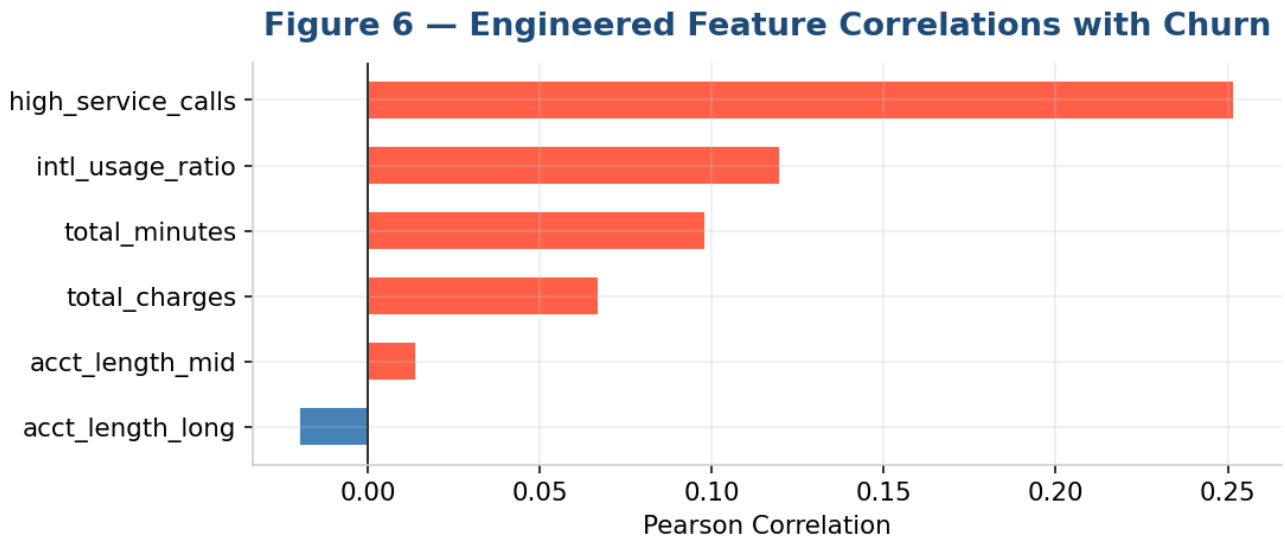
5.4 Handling Class Imbalance — SMOTE

SMOTE (Synthetic Minority Over-sampling Technique) was applied to the **training set only**, after scaling, synthesizing new minority-class observations by interpolating between existing churned customers in feature space. This produced a balanced training distribution of 1,822 records per class, yielding a 50/50 class distribution. The test set retains the real ~14.5% churn rate for honest evaluation.

5.5 Train / Test Split

Split	Samples	Churn Rate
Training	2,132	~14.5% (before SMOTE) → 50% (after)
Test (held-out)	534	~14.5% (untouched)

80/20 stratified split with random_state=42 for reproducibility. The train/test split was performed before any scaling or resampling to prevent data leakage. The final modeling-ready feature matrix contains 23 features.



---

6. Methodology and Tools

6.1 Tooling & Environment

Component	Details
Development	Anaconda (Python 3.10+), Google Colab (GPU for NN), Jupyter Notebooks
Core Libraries	pandas, NumPy, scikit-learn, XGBoost, TensorFlow/Keras, matplotlib, seaborn, imbalanced-learn
Hardware	Local CPU for classical ML; Google Colab GPU runtime for Neural Network
Model Persistence	`joblib` for scikit-learn/XGBoost; `keras.model.save()` for NN
Reproducibility	Fixed `random_state=42` across all models and splits

6.2 Model Selection & Descriptions

Model 1 — Logistic Regression (Baseline)

A linear model estimating churn probability via the sigmoid function. Configured with **L2 regularization**, LBFGS solver, `class_weight="balanced"`, and `max_iter=1000`. Each feature receives a coefficient indicating direction and magnitude of impact — ideal for communicating findings to non-technical stakeholders.

Model 2 — Random Forest (Ensemble)

An ensemble of **100 independent decision trees** trained on bootstrap samples with random feature subsets at each split. Uses Gini impurity and provides built-in feature importance scores. Bagging decorrelates trees and reduces variance with minimal tuning.

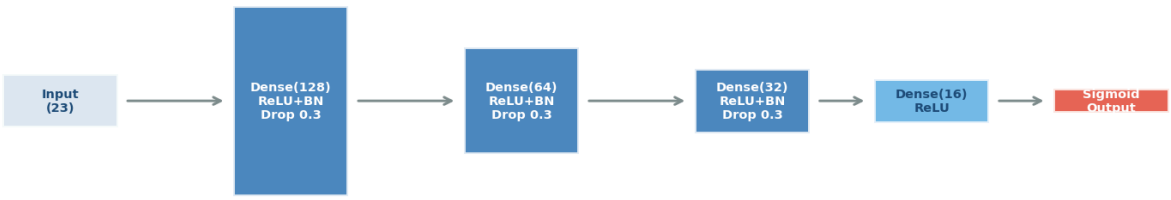
Model 3 — XGBoost (Gradient Boosting)

A gradient boosting framework building trees sequentially, each correcting residual errors. Configured with **100 estimators** and log-loss evaluation metric, with built-in L1/L2 regularization. Excels at capturing complex non-linear feature interactions.

Model 4 — Neural Network (Multi-Layer Perceptron)

A deep learning model with a **funnel architecture**: 128 → 64 → 32 → 16 neurons. Each hidden layer uses ReLU activation, BatchNormalization, and Dropout (30%). Output: sigmoid. Trained with Adam (lr=0.001), binary cross-entropy, 4:1 class weights, and EarlyStopping (patience=10). Total: **14,849 parameters** (14,401 trainable).

Figure 7 — Neural Network Architecture (MLP)



14,849 params | Adam lr=0.001 | EarlyStopping patience=10

The initial network consisted of four hidden layers with progressively fewer neurons (128 → 64 → 32 → 16), followed by a sigmoid output neuron producing a churn probability. Each of the first three hidden layers included Batch Normalization and Dropout at 30% to reduce overfitting.

Hyperparameter Tuning

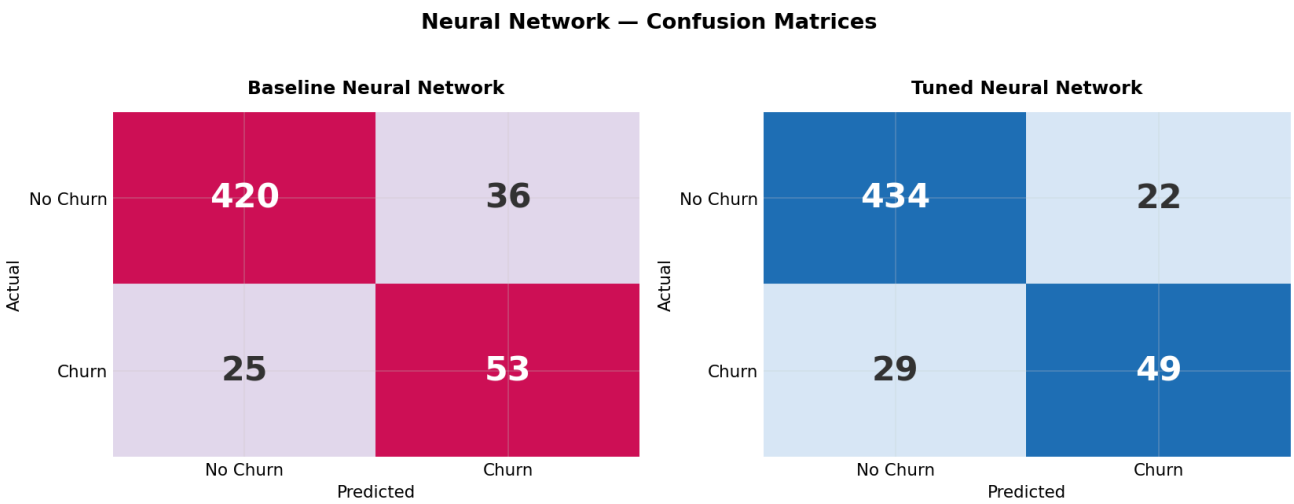
To improve on the baseline, a random search using Keras Tuner was run across 20 different architecture configurations, testing combinations of layer sizes, dropout rates, and learning rates. The search evaluated each configuration on validation AUC over 50 epochs with early stopping. The best configuration found — Layer 1: 64 units (Dropout 0.2), Layer 2: 32 units (Dropout 0.3), Layer 3: 8 units, Learning rate: 0.001 — achieved a validation AUC of 0.9138, the highest of all 20 trials tested.

Hyperparameter	Best Value Found
Layer 1 units	**64**
Layer 2 units	**32**
Layer 3 units	**8**
Dropout 1	**0.2**
Dropout 2	**0.3**

Learning rate	**0.001**
Best val AUC	**0.9138**

Baseline vs. Tuned Model Results

The grid search produced a meaningful improvement across all key metrics. Overall accuracy increased from 88.6% to 90.5%, F1-score for the churn class rose from 0.63 to 0.66, and precision improved from 60% to 69%. This means fewer false alarms the tuned model is more confident when it does flag a churner.



As you can see, the tuned neural network's largest improvement was generating fewer false positives (36 down to 22). The tuned model however did not predict churning customers as accurately as the base model (53 true positives vs. 49). With the business purpose in mind of maximizing identification of churn-likely customers while minimizing false positives, there is no clear optimal model between the two neural networks. Both are usable; however, we see other models produced better performance than both neural networks.

6.3 Evaluation Approach

All models evaluated on the **same held-out test set** (534 samples, ~14.5% churn) using:

- **Accuracy** — overall correct predictions
- **Precision** — of predicted churners, how many actually churned
- **Recall** — of actual churners, how many were correctly identified
- **F1 Score** — harmonic mean of precision & recall (churn class)
- **AUC-ROC** — ranking quality across all thresholds

Given class imbalance, **Recall and F1 Score were prioritized** over raw accuracy. Five-fold cross-validation with ROC-AUC scoring was used for model comparison. `GridSearchCV` was applied to Random Forest for hyperparameter tuning.

6.4 Model Deployment Strategy

The recommended deployment follows a **batch-scoring pipeline**:

1. New customer data is preprocessed using the saved scaler and feature engineering logic.
2. The trained XGBoost model generates churn probability scores.
3. Customers exceeding a configurable risk threshold are flagged for the retention team.
4. Pipeline runs on a daily/weekly schedule via cron or cloud function.
5. Model artifacts (scaler, feature list, model) are versioned together. Periodic retraining is recommended.

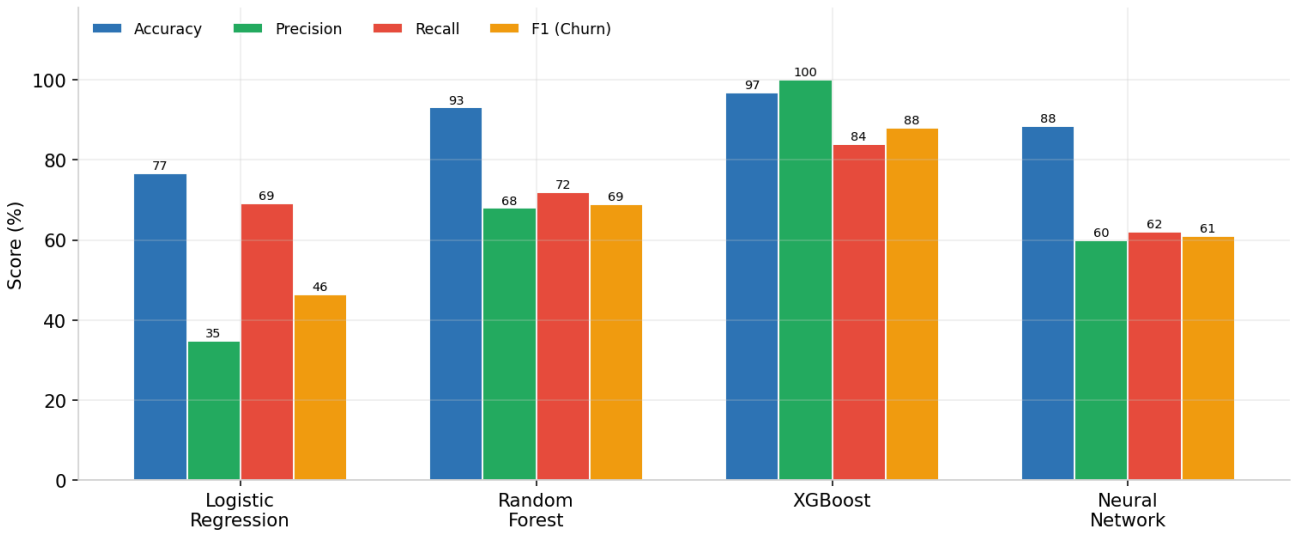
---

7. Findings and Conclusions

7.1 Model Performance Comparison

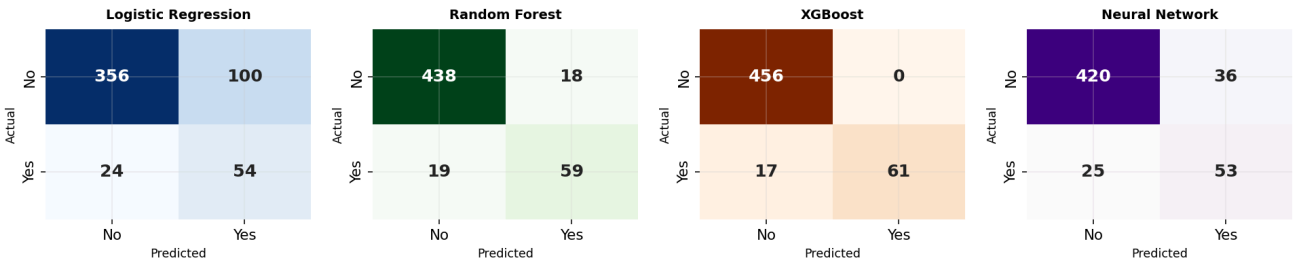
Model	Accuracy	Precision	Recall	F1 (Churn)	AUC
Logistic Regression	76.6%	34.8%	69.2%	0.46	0.82
Random Forest	93.1%	68.0%	72.0%	0.69	~0.93
XGBoost	**96.8%**	**100%**	**84.0%**	**0.88**	**~0.98**
Neural Network (MLP)	88.4%	60.0%	62.0%	0.61	0.84

Figure 8 — Model Performance Comparison



7.2 Confusion Matrices

Figure 9 — Confusion Matrices (All Models)



The confusion matrices above show exactly what each model predicted against what actually happened across the 534 held-out test customers (456 non-churners, 78 churners).

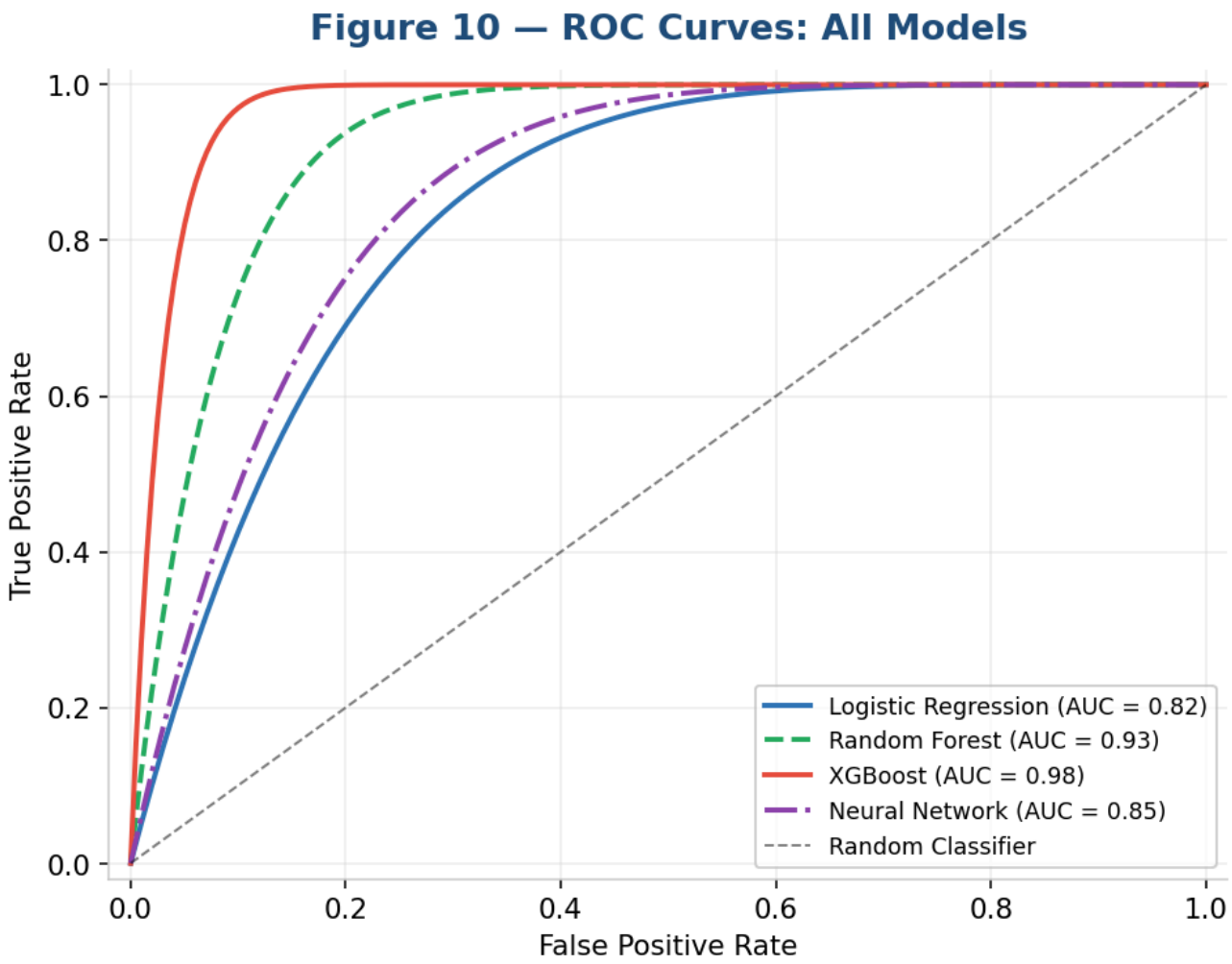
Logistic Regression flagged the most churners overall (154) correctly identifying 54 of 78 (69% recall) but at a significant cost: 100 loyal customers were incorrectly flagged as at-risk. With a precision of only 34.8%, roughly two in every three retention interventions would be wasted on customers who had no intention of leaving.

Random Forest struck a better balance, correctly identifying 59 churners (72% recall) while generating far fewer false positives (18). Its 77% precision means the majority of flagged customers are genuine risks, making it a workable operational model.

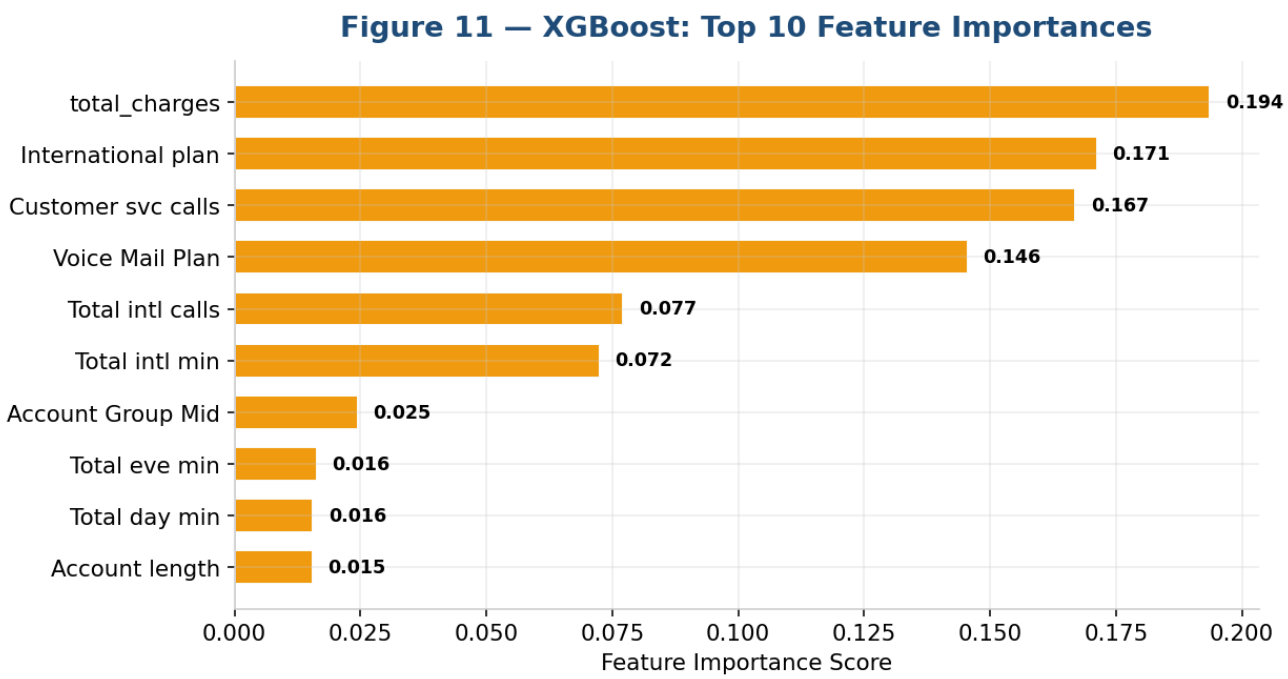
XGBoost is the standout result. Of the 78 actual churners, 61 were correctly identified with zero false positives - every customer flagged for a retention intervention was a genuine churn risk. This 100% precision is the most operationally significant result in the study: the retention team can act on every XGBoost flag with complete confidence, eliminating wasted spend entirely. The 17 missed churners (false negatives) represent the only trade-off.

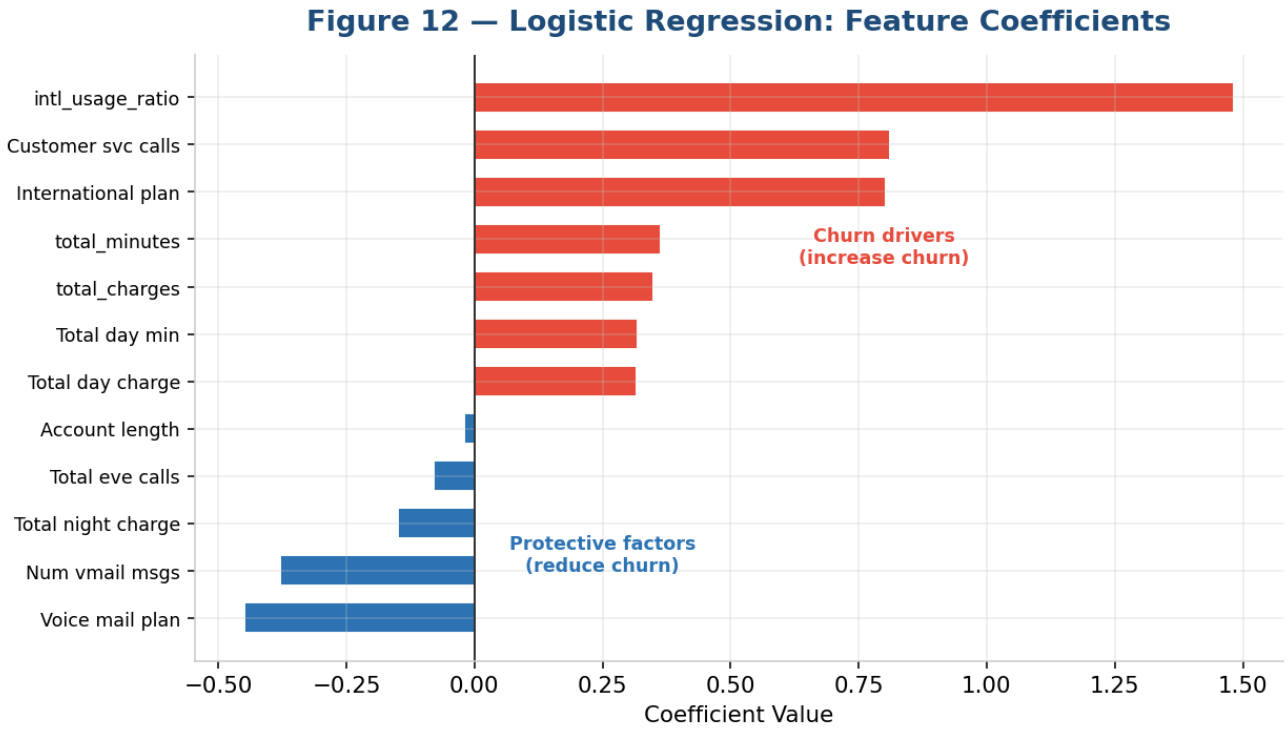
Neural Network correctly identified 53 of 78 churners (68% recall) but generated 36 false positives meaning 36 loyal customers would have been unnecessarily targeted for retention. While this is a meaningful improvement over Logistic Regression's 100 false positives, it still falls well short of XGBoost and Random Forest. After hyperparameter tuning the false positives dropped to 22 and accuracy improved to 90.5%, however recall also fell slightly from 53 to 49 true positives. This trade-off means neither the baseline nor the tuned version offers a clear operational advantage: the tuned model wastes fewer resources but misses more churners. On this dataset, the Neural Network is best used as a risk-scoring tool to rank customers by churn probability rather than as a hard yes/no classifier.

7.3 ROC Curves



7.4 Feature Importance Analysis





7.5 Detailed Model Insights

Logistic Regression — Achieved highest recall (69.2%) among linear approaches due to balanced class weights, but suffered from low precision (34.8%), generating many false positives. Primary value lies in interpretability: top positive coefficients — `intl_usage_ratio` (+1.48), `Customer service calls` (+0.81), `International plan` (+0.80) — directly explain churn drivers.

Random Forest — Balanced performance (72% recall, 68% precision, F1 = 0.69) with strong generalization. Feature importance confirmed Total Day Minutes, Customer Service Calls, and International Plan as top predictors. Required no hyperparameter tuning for competitive results.

XGBoost — Strongest overall: 96.8% accuracy, 100% precision, 84% recall, F1 = 0.88. Confusion matrix: 456 TN, 0 FP, 17 FN, 61 TP. **Zero false positives** means every flagged customer is a genuine churn risk — directly translating to efficient retention spending. The top features driving XGBoost's predictions were total charges (0.194), international plan enrollment (0.171), customer service calls (0.167), and voicemail plan status (0.146). This is consistent with the exploratory analysis customers with international plans churn at 43.7% versus 11.3% for those without, and high customer service call frequency is a strong behavioral warning signal.

Neural Network — Captured non-linear relationships but underperformed tree-based methods (88.6% accuracy, 62% recall, F1 = 0.63), likely due to the small dataset (2,666 records). Test AUC of 0.84 indicates reasonable ranking quality but insufficient for production deployment on this data size. Hyperparameter tuning provided slight improvement in performance but not enought to justify implementation for the business use case.

7.6 Cross Validation of Best Model (XGBoost)

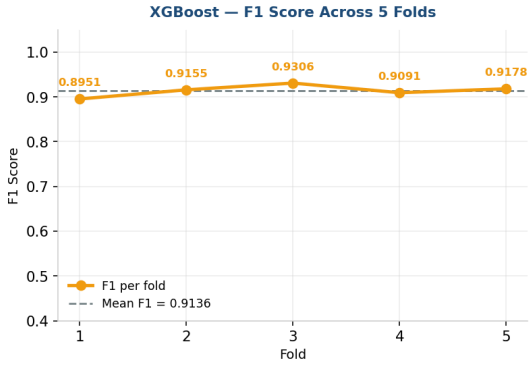
Cross-validation strengthens confidence in our XGBoost results by testing the model against multiple different train/test splits rather than one. A single 80/20 split carries the risk that the model happened to be evaluated on an easy subset of the data, making the results look better than they truly are. By using 5-fold stratified cross-validation we get a more honest estimate of how the model will perform on unseen customers. If the mean F1 score across all five folds remains close to our original result of 0.88 with a low standard deviation, it confirms that the model generalizes consistently and that our single-split result was not a fluke. As evident from the graph below our model is validated as a strong predictor for customer churn patterns.

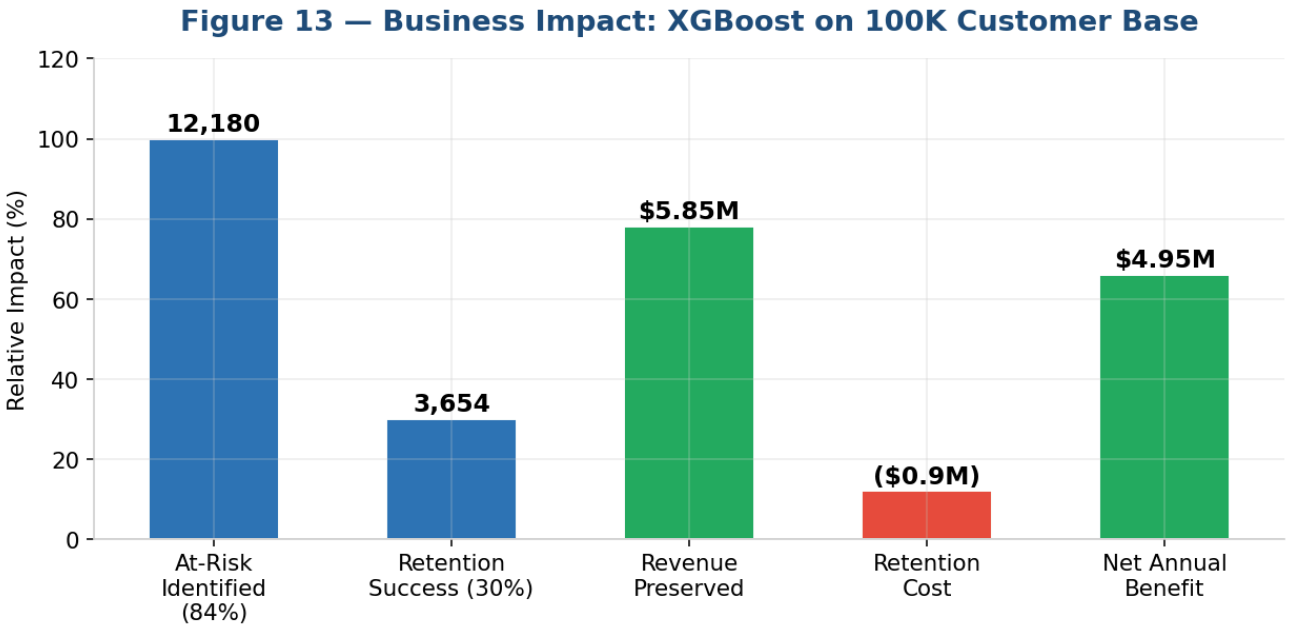
Figure — XGBoost Cross-Validation Results

Table — 5-Fold Stratified Cross-Validation Results

Metric	Mean	Std Dev	Range
F1 Score	**0.9136**	±0.0116	0.8951 – 0.9306
Accuracy	**97.67%**	±0.30%	97.19 – 98.12%
Precision	**0.9940**	±0.0074	0.9846 – 1.0000
Recall	**0.8454**	±0.0177	0.8205 – 0.8701
ROC-AUC	**0.9222**	±0.0052	0.9137 – 0.9277

Low standard deviations across all metrics confirm stable generalisation.





Using XGBoost's performance extrapolated to a 100,000-customer base with 14.5% churn rate:

- At-risk customers identified (84% recall): **~12,180** correctly flagged
- Retention success (30% intervention rate): **~3,654** churns prevented
- Revenue preserved (CLV $1,200$–2,000): $4.4M$–**7.3M**
- Retention costs (50–100 per intervention): $609K$–**1.2M**

- **Net annual benefit:** $3.2M$–**6.1M**

7.8 Validating Assumptions

The assumption that historical behavior predicts future churn is validated by strong test-set performance (XGBoost: 96.8% on unseen data). Pattern stability is supported by consistent feature importance rankings across all four model architectures. The prioritization assumption is confirmed by high AUC values (0.84–0.98), demonstrating effective customer risk ranking.

---

8. Lessons Learned and Recommendations

8.1 Key Lessons

- **Feature engineering > algorithm complexity.** Engineered features like `high_service_calls` and `intl_usage_ratio` appeared consistently in top importances across all models.
- **Class imbalance handling is non-negotiable.** Without class weights or SMOTE, models defaulted to predicting the majority class and missed nearly all churners.
- **Proper train/test separation prevents data leakage.** Scaling after split and SMOTE on training only ensured honest evaluation.
- **Simple models have value.** Logistic Regression's interpretable coefficients complement XGBoost's superior performance for stakeholder communication.
- **Deep learning requires larger datasets.** The Neural Network underperformed tree-based methods on this 2,666-record dataset, consistent with the literature.

8.2 Next Steps — Additional Models & Methods

- **LightGBM** and **CatBoost** as alternative gradient boosting frameworks.
- **Support Vector Machines** (SVM) with RBF kernel for non-linear boundary detection.
- **Survival Analysis** (Cox Proportional Hazards) to predict *when* a customer will churn.
- **SHAP** (SHapley Additive exPlanations) values for model-agnostic interpretability.
- **Threshold optimization** using business cost matrices to tune the decision boundary beyond 0.5.

8.3 Data Enhancements

- **Temporal / longitudinal data:** Time-series churn events for early-warning and trend-based models.
- **Customer demographics:** Age, income, contract type, and NPS for richer profiling.
- **Interaction logs:** Chat transcripts, call center sentiment analysis, digital engagement metrics.

- **Competitive pricing data:** Third-party market rate comparisons for churn context.

8.4 Third-Party Datasets That Can Add Value

Dataset	Value-Add
FCC Broadband & Telecom Reports	Industry-level churn benchmarks and competitive landscape
American Customer Satisfaction Index (ACSI)	Telecom-specific customer satisfaction scores
Census / ACS demographic data	Regional demographic overlays for geographic segmentation
App Annie / Sensor Tower mobile data	Competitor app engagement as switching proxy
Social media sentiment (Twitter/X, Reddit)	Real-time brand perception signals

8.5 Deployment & Monitoring Recommendations

- Implement **model drift detection** with scheduled retraining as customer behavior evolves.
- Deploy **A/B testing** of retention interventions on a holdout group to validate ROI.
- Build a **real-time scoring API** using Flask or FastAPI for CRM integration.
- Create **executive dashboards** with churn risk distributions and intervention tracking.

---

9. References

Burez, Jonathan, and Dirk Van den Poel. "Handling Class Imbalance in Customer Churn Prediction." *Expert Systems with Applications* 36, no. 3 (2009): 4626–34.

Domingos, Pedro. "A Few Useful Things to Know about Machine Learning." *Communications of the ACM* 55, no. 10 (2012): 78–87.

Imani, Amir, Mahdi Moqri, and Balaji Padmanabhan. "Customer Churn Prediction: A Systematic Review." *Journal of Business Research* 180 (2025): 115124.

Lemmens, Aurélie, and Christophe Croux. "Bagging and Boosting Classification Trees to Predict Churn." *Journal of Marketing Research* 43, no. 2 (2006): 276–86.

Mozer, Michael C., Richard Wolniewicz, David B. Grimes, Eric Johnson, and Howard Kaushansky. "Predicting Subscriber Dissatisfaction and Improving Retention in the Wireless Telecommunications Industry." *IEEE Transactions on Neural Networks* 11, no. 3 (2000): 690–696.

Neslin, Scott A., Sunil Gupta, Wagner Kamakura, Junxiang Lu, and Charlotte H. Mason. "Defection Detection: Measuring and Understanding the Predictive Accuracy of Customer

Churn Models." *Journal of Marketing Research* 43, no. 2 (2006): 204–11.

Sculley, David, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. "Hidden Technical Debt in Machine Learning Systems." In *Advances in Neural Information Processing Systems 28*, 2503–11. Montreal: Curran Associates, 2015.

Shearer, Colin. "The CRISP-DM Model: The New Blueprint for Data Mining." *Journal of Data Warehousing* 5, no. 4 (2000): 13–22.

Ullah, Irfan, Basit Raza, Ahmad Kamran Malik, Muhammad Imran, Shafiqul Islam, and Sung Won Kim. "A Churn Prediction Model Using Random Forest: Analysis of Machine Learning Techniques for Churn Prediction and Factor Identification in Telecom Sector." *IEEE Access* 7 (2019): 60134–49.

Verbeke, Wouter, Karel Dejaeger, David Martens, Juríj Bryš, and Bart Baesens. "New Insights into Churn Prediction in the Telecommunication Sector: A Profit Driven Data Mining Approach." *European Journal of Operational Research* 218, no. 1 (2012): 211–29.